C-PROGRAMING (CSA0202)

CO2 ASSIGNMENT – 2

NAME: LEELAVATHI R

REG.NO:1925260191

Q1 – Precedence, Associativity and the Discount Bug

The original pharmacy expression is:

```
total = price * qty - price * qty * 15 / 100 * senior
        + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total;
count = count++ + 1;
```

Predictions before execution

Assume price and qty are integers, senior is 1 for a senior citizen and 0 otherwise, and all calculations use integer arithmetic. Choose these three examples:

Bill	price	qty	senior	Original total	Original net
A	100	2	1	180	180
B	250	5	0	1312	1262
C	50	4	1	180	180

For Bill A, gross = 100 × 2 = 200. The discount term is evaluated as 200 × 15 / 100 × 1 = 30, and the tax term as 200 × 5 / 100 = 10. Therefore the written expression gives 200 − 30 + 10 = 180. Since 180 is not greater than 1,000, net remains 180.

For Bill B, gross = 1,250. Since senior = 0, the discount term becomes zero. The tax term is 1,250 × 5 / 100 = 62, because integer division truncates the fractional part. total = 1,312. The conditional operator subtracts 50 because total > 1,000, so net = 1,262.

For Bill C, gross = 200. The discount term is 30 and tax term is 10, giving 180. net remains 180.

Operator order and errors

- Multiplication, division and remainder have higher precedence than addition and subtraction. Operators at the same precedence level are evaluated left to right.
- The multiplication and division in each discount/tax term are grouped left to right. Integer division discards the fractional part, so 1,250 × 5 / 100 produces 62 rather than 62.5.
- The original expression adds GST on the gross amount, not on the discounted amount. The intended calculation is usually gross − discount + GST on the discounted amount.
- The expression has no explicit brackets or intermediate values, making the intended tax base difficult to see.
- The conditional operator ?: has lower precedence than relational >. Thus total > 1000 is tested first and the selected result is assigned to net.
- count = count++ + 1 modifies count twice without a sequencing point between the modifications. This is undefined behaviour in C; the compiler is not required to produce a predictable value. GCC with -Wall may issue a sequence-point or unsequenced-operation warning, depending on compiler version and warning settings.

Correct and readable design

```
#include <stdio.h>

int main(void) {

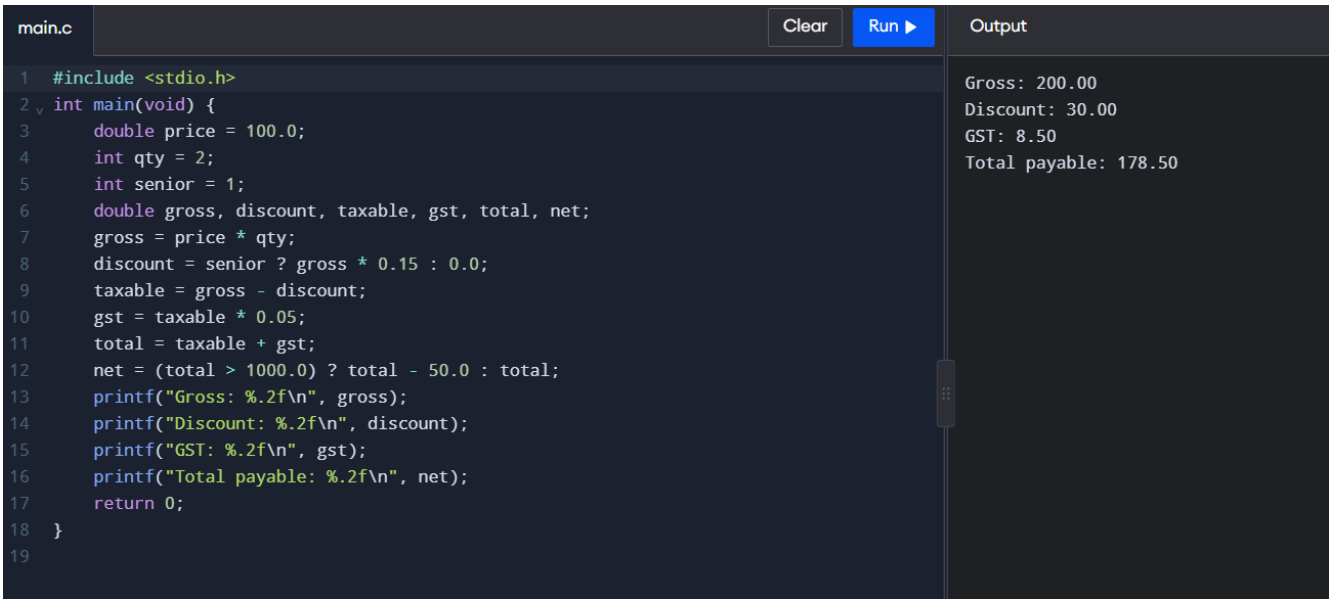
    double price = 100.0;

    int qty = 2;

    int senior = 1;

    double gross, discount, taxable, gst, total, net;
```

```
    gross = price * qty;
    discount = senior ? gross * 0.15 : 0.0;
    taxable = gross - discount;
    gst = taxable * 0.05;
    total = taxable + gst;
    net = (total > 1000.0) ? total - 50.0 : total;
    printf("Gross: %.2f\n", gross);
    printf("Discount: %.2f\n", discount);
    printf("GST: %.2f\n", gst);
    printf("Total payable: %.2f\n", net);
    return 0;
}
```



For Bill A, the corrected calculation is gross Rs 200.00, discount Rs 30.00, taxable amount Rs 170.00, GST Rs 8.50 and final amount Rs 178.50. In a real billing system, currency should preferably be stored as integer paise and rounded according to a documented rule.

To increment count safely, write count = count + 1; or count++; as a separate statement. Do not combine multiple modifications of the same variable in one unsequenced expression.

Alternative: intermediate variables and complexity

Using gross, discount, taxable, GST and total creates a few temporary variables. The number of arithmetic operations remains constant, so time complexity is O(1). The additional storage is O(1), because only a fixed number of variables is used. The small memory cost is worthwhile because each stage can be checked, tested and printed independently. Brackets should still be used to show the intended order.

Q2 – When Does Short-Circuit Evaluation Matter?

The concession condition is: (senior citizen AND eligible class) OR disability card verified. The server verification is assumed to be the slow function.

```
#include <stdio.h>
int server_result_for(int passenger_id)
{
   return 1;
}
int verify_disability_card(int passenger_id)
{
   return server_result_for(passenger_id);
}
int concession(int senior, int eligible, int passenger_id)
{
   return (senior && eligible) ||
         verify_disability_card(passenger_id);
}
int main(void)
{
   int result = concession(1, 1, 101);

   if (result)
      printf("Passenger is eligible for concession.\n");
   else
      printf("Passenger is not eligible for concession.\n");
   return 0;
}
```

```c
#include <stdio.h>
int server_result_for(int passenger_id)
{
    return 1;
}
int verify_disability_card(int passenger_id)
{
    return server_result_for(passenger_id);
}
int concession(int senior, int eligible, int passenger_id)
{
    return (senior && eligible) ||
           verify_disability_card(passenger_id);
}
int main(void)
{
    int result = concession(1, 1, 101);

    if (result)
        printf("Passenger is eligible for concession.\n");
    else
        printf("Passenger is not eligible for concession.\n");
    return 0;
}
```

```
Passenger is eligible for concession.
```

Short-circuit behaviour

- In A && B, if A is false, B is not evaluated because the whole expression must be false.
- In A || B, if A is true, B is not evaluated because the whole expression must be true.
- The operators & and | evaluate both operands when used on integer conditions. They do not provide short-circuit behaviour.

A useful experiment is to place a counter increment inside the verification function and print it after evaluating a set of passengers. For the original order, the function is called only when the senior-and-eligible condition is false.

Call-count experiment for 1,000 passengers

For a measurable example, assume 300 passengers are both senior and eligible, 200 are senior but in an ineligible class, and 500 are not senior. With the expression (senior && eligible) || verify_card(), verification is skipped for the 300 passengers whose first combined condition is true. It is called for the remaining 700.

Condition order / implementation	Verification calls
(senior && eligible) || verify_card()	700
verify_card() || (senior && eligible)	1,000
(senior & eligible) | verify_card()	1,000

The first order avoids 300 slow calls in this sample. The second always calls the server first; if the verification returns true, the second part is skipped, but the server call has already happened. The third uses bitwise operators and evaluates both sides.

The cheapest condition should generally be placed first in an && chain when it is likely to be false, and a cheap condition likely to be true should be placed first in an || chain. However, the conditions can only be reordered when doing so preserves the required meaning and has no important side effects.

Changed condition: verification writes an audit log

If verification must record every passenger, short-circuiting is no longer acceptable for that audit requirement. Call verify_disability_card() explicitly for every passenger, store its result, and then combine the values. This guarantees one audit record per passenger and 1,000 calls for 1,000 passengers. The audit requirement is more important than saving calls.

```
int verified = verify_disability_card(passenger_id); /* always called */
int concession_granted = (senior && eligible) || verified;
```

Nested if and complexity

A nested-if version can preserve short-circuiting: first test senior; then eligible; only if the first path does not grant the concession, call verification. Its number of verification calls is the number of passengers who do not satisfy senior AND eligible. For n passengers, worst-case time is O(n), with up to n server calls. Using & and | also makes up to n calls. The explicit audit version intentionally makes exactly n calls. The time cost of the remote service usually dominates the local Boolean operations.

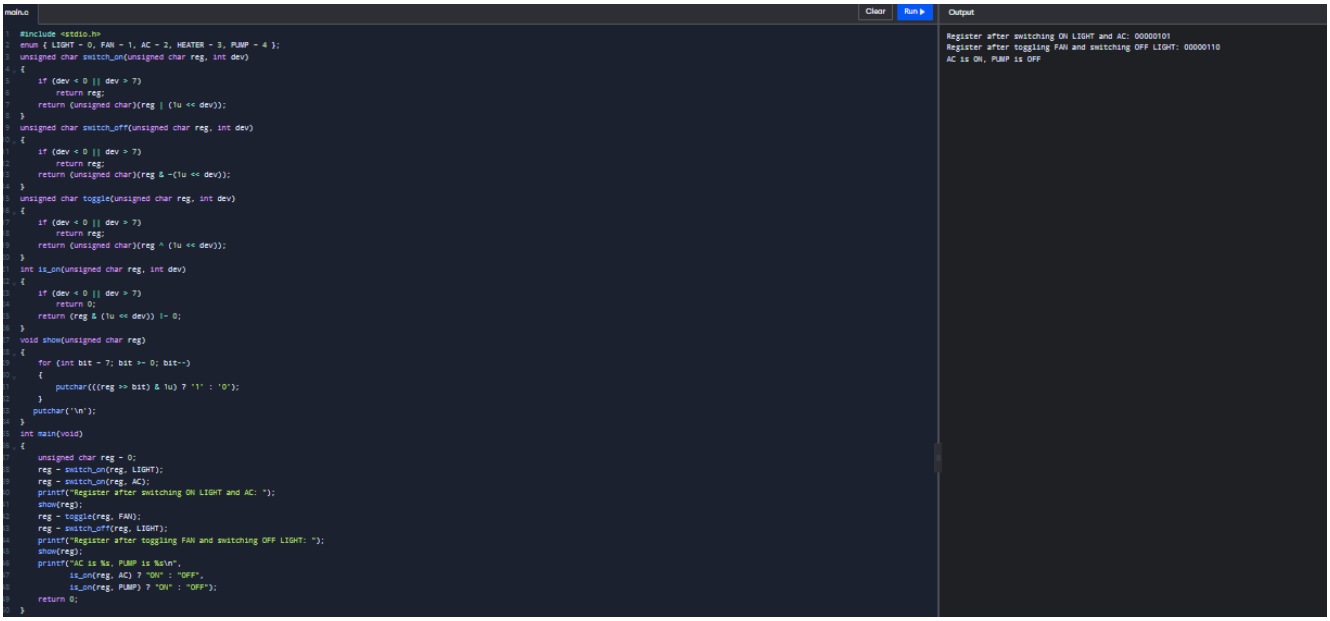
Q3– Engineer a Device-Status Register with Bitwise Operators

One unsigned char stores eight device states. Bit 0 represents LIGHT, bit 1 FAN, bit 2 AC, bit 3 HEATER and bit 4 PUMP. A set bit means ON and a cleared bit means OFF.

Completed functions

```
#include <stdio.h>
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };
unsigned char switch_on(unsigned char reg, int dev)
{
   if (dev < 0 || dev > 7)
      return reg;
   return (unsigned char)(reg | (1u << dev));
}
unsigned char switch_off(unsigned char reg, int dev)
{
   if (dev < 0 || dev > 7)
      return reg;
   return (unsigned char)(reg & ~(1u << dev));
}
unsigned char toggle(unsigned char reg, int dev)
{
   if (dev < 0 || dev > 7)
      return reg;
   return (unsigned char)(reg ^ (1u << dev));
}
int is_on(unsigned char reg, int dev)
{
   if (dev < 0 || dev > 7)
      return 0;
   return (reg & (1u << dev)) != 0;
}
void show(unsigned char reg)
{
   for (int bit = 7; bit >= 0; bit--)
   {
      putchar(((reg >> bit) & 1u) ? '1' : '0');
   }
  putchar('\n');
```

```
}
int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    printf("Register after switching ON LIGHT and AC: ");
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    printf("Register after toggling FAN and switching OFF LIGHT: ");
    show(reg);
    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```



trace and predicted output

Statement	Register (binary)	Hexadecimal
Initial reg = 0	00000000	0x00
Switch LIGHT on	00000001	0x01
Switch AC on	00000101	0x05
Toggle FAN	00000111	0x07
Switch LIGHT off	00000110	0x06

Expected output:

```
00000101
00000110
AC is ON, PUMP is OFF
```

Bitwise explanations

- switch_on uses OR with a mask. OR sets the selected bit to 1 without changing other bits.
- switch_off uses AND with the complement of a mask. The selected bit becomes 0 and all other bits remain unchanged.
- toggle uses XOR. XOR with 1 flips the selected bit.
- is_on uses AND to test whether the selected bit is set.

The expression reg & 1 << dev is interpreted as reg & (1 << dev), because shift has higher precedence than bitwise AND. It tests the selected bit. The expression (reg & 1) << dev first tests bit 0 and then shifts that result; it does not test bit dev. These expressions are not equivalent.

Use unsigned char rather than signed char for a bit register. If bit 7 is set in a signed char, the value may become negative. Right-shifting a negative signed value is implementation-defined on many C implementations and can cause sign extension. Unsigned types provide well-defined logical right-shift behaviour.

Array alternative and complexity

With char state[8], each device has a separate element containing 0 or 1. Setting, clearing, toggling and testing one device are O(1). Counting ON devices requires scanning all eight entries, which is O(8), effectively O(1) for a fixed eight-device system. For 1,000,000 homes, one byte per device requires 8,000,000 bytes, while one 8-bit register per home requires 1,000,000 bytes. The bit register is eight times more compact, although array elements can be easier to understand and extend.

Q4 – else–if Ladder or switch?

The grade rules are S for 90–100, A for 80–89, B for 70–79, C for 60–69, D for 50–59, F below 50, and '?' for invalid marks.

Implementation 1: else-if ladder

```
char grade_if(int marks) {
    if (marks < 0 || marks > 100) return '?';
    if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}
```

Implementation 2: switch on marks / 10

```
char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {
        case 10:
        case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
```

The range check must occur before switch. Integer division groups marks into tens: 89 / 10 is 8, 90 / 10 is 9 and 100 / 10 is 10. switch compares one integral expression against constant case labels; it cannot directly express a range such as 75–84.

Boundary test set

Marks	Expected grade	Reason
100	S	Upper valid limit
90	S	S lower boundary
89	A	Immediately below S
80	A	A lower boundary
79	B	Immediately below A
70	B	B lower boundary
69	C	Immediately below B
60	C	C lower boundary
59	D	Immediately below C
50	D	D lower boundary
49	F	Below D
0	F	Lowest valid mark
-1	?	Invalid negative mark
101	?	Invalid mark above 100

A missing break in a switch causes fall-through: execution continues into the following case labels until a break or return is reached. In this implementation each case returns immediately, so removing a break has no effect

because there is no break to remove. If the cases used assignments instead of return, a missing break could overwrite the grade with a later case's value.

Scholarship classification and lookup table

A nested if or a switch inside an if is suitable when scholarship rules depend on both grade and income bands. A lookup table is preferable when there are many fixed grade/income combinations, because it centralizes the policy and is easy to test. The exact scholarship thresholds are not supplied in the question, so they must be obtained from the university before implementing that part.

```
char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
char result;
if (marks < 0 || marks > 100)
    result = '?';
else
    result = grades[marks / 10];
```

The lookup table takes O(1) time per grade and uses 11 bytes. A ladder may perform up to k comparisons for k grade ranges, giving O(k) time; for six fixed ranges this is still small. The table is concise but needs careful validation of its index.

```
#include <stdio.h>

char grade_if(int marks)
{
   if (marks < 0 || marks > 100)
      return '?';
   if (marks >= 90)
      return 'S';
   else if (marks >= 80)
      return 'A';
   else if (marks >= 70)
      return 'B';
   else if (marks >= 60)
      return 'C';
   else if (marks >= 50)
      return 'D';
   else
      return 'F';
}
char grade(int marks)
{
   if (marks < 0 || marks > 100)
      return '?';
   switch (marks / 10)
   {
      case 10:
```

```
        case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}

char grade_array(int marks)
{
    char grades[11] = {
        'F', 'F', 'F', 'F', 'F',
        'D', 'C', 'B', 'A', 'S', 'S'
    };

    if (marks < 0 || marks > 100)
        return '?';
    else
        return grades[marks / 10];
}
int main()
{
    int marks;
    printf("Enter marks: ");
    scanf("%d", &marks);
    printf("Grade using if-else: %c\n", grade_if(marks));
    printf("Grade using switch: %c\n", grade(marks));
    printf("Grade using array: %c\n", grade_array(marks));
    return 0;
}
```

main.c

Clear Run ▶

```c
#include <stdio.h>
char grade_if(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';
    if (marks >= 90)
        return 'S';
    else if (marks >= 80)
        return 'A';
    else if (marks >= 70)
        return 'B';
    else if (marks >= 60)
        return 'C';
    else if (marks >= 50)
        return 'D';
    else
        return 'F';
}
char grade(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';
    switch (marks / 10)
    {
        case 10:
        case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}

```

Output

```
Enter marks: 90
Grade using if-else: S
Grade using switch: S
Grade using array: S
```

Q5 – Count the Work Done by a Loop

A composite number has at least one divisor no greater than its square root. Therefore it is sufficient to test divisors while i × i <= n. The program should stop as soon as a divisor is found.

Completed function using for

```
unsigned long checks = 0;
int is_prime(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i * i <= n; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}
```

For very large unsigned long values, i * i may overflow. A safer condition is i <= n / i, which expresses the same limit without multiplying.

```
int is_prime_safe(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}
```

while and do-while versions

```
int is_prime_while(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    unsigned long i = 2;
    while (i <= n / i) {
        checks++;
        if (n % i == 0) return 0;
        i++;
    }
    return 1;
}

int is_prime_do(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    unsigned long i = 2;
    do {
        if (i > n / i) break;
        checks++;
        if (n % i == 0) return 0;
        i++;
    } while (1);
    return 1;
}
```

The do-while loop executes its body at least once, so it needs a guard before attempting a divisor. The early n < 2 check handles 0 and 1. The condition i > n / i prevents an unnecessary test for n = 2 or n = 3.

Division counts

Ticket number	Result	Divisions with i <= n/i
97	Prime	8 (2 through 9)
100	Not prime	1 (divisor 2 found immediately)
7919	Prime	87 (2 through 88)
1,000,003	Prime	999 (2 through 1000)

The question's expected counts correspond to testing all candidate integers from 2 through the square-root boundary, counting each modulus/divisibility test. Changing the condition to i < n tests almost every possible divisor and greatly increases work. For example, 97 requires 95 tests (2 through 96), 100 still stops after one test, 7919 requires 7917 tests, and 1,000,003 requires 1,000,001 tests.

The improved trial-division algorithm has $O(\sqrt{n})$ time per prime query and O(1) extra space. The original loop up to $n - 1$ has O(n) time. For repeated queries up to a fixed N, the Sieve of Eratosthenes marks composites once in O(N log log N) time and uses O(N) space; each later primality query is O(1).

The sieve is useful when many tickets within a known limit must be checked. Trial division is simpler and uses much less memory for only a few large values.

Q6 – Loops That Must Terminate: Loan Repayment Schedule

A while loop is appropriate because the number of months is not known in advance. Each month, interest is added to the balance and a repayment is subtracted. The loop continues while the outstanding balance is positive.

Using balance != 0 with floating-point values is dangerous because repeated decimal calculations may leave a tiny positive or negative residue. A better approach is to store money in integer paise and use balance_paise > 0 as the condition.

Loan program using integer paise

```
#include <stdio.h>
int main(void) {
    long long balance = 1000000;
    long long repayment = 200000;
    long long total_interest = 0;
    int month = 0;
    const int MAX_MONTHS = 360;
    while (balance > 0) {
        if (month >= MAX_MONTHS) {
            printf("Safety limit reached; loan is not fully repaid.\n");
            break;
        }
        long long interest = (balance * 2 + 50) / 100;
        balance += interest;
        total_interest += interest;
        long long actual_payment =
            (repayment < balance) ? repayment : balance;
        balance -= actual_payment;
        month++;
        printf("%d interest=Rs %lld.%02lld payment=Rs %lld.%02lld balance=Rs %lld.%02lld\n",
               month,
               interest / 100, interest % 100,
               actual_payment / 100, actual_payment % 100,
               balance / 100, balance % 100);
    }
    printf("Months: %d\n", month);

    printf("Total interest: Rs %lld.%02lld\n",
           total_interest / 100,
           total_interest % 100);
```

return 0;

}

```c
#include <stdio.h>
int main(void) {
    long long balance = 1000000;
    long long repayment = 200000;
    long long total_interest = 0;
    int month = 0;
    const int MAX_MONTHS = 360;
    while (balance > 0) {
        if (month >= MAX_MONTHS) {
            printf("Safety limit reached; loan is not fully repaid.\n");
            break;
        }
        long long interest = (balance * 2 + 50) / 100;
        balance += interest;
        total_interest += interest;
        long long actual_payment =
            (repayment < balance) ? repayment : balance;
        balance -= actual_payment;
        month++;
        printf("%d interest=Rs %lld.%02lld payment=Rs %lld.%02lld balance=Rs
%lld.%02lld\n",
               month,
               interest / 100, interest % 100,
               actual_payment / 100, actual_payment % 100,
               balance / 100, balance % 100);
    }
    printf("Months: %d\n", month);

    printf("Total interest: Rs %lld.%02lld\n",
           total_interest / 100,
           total_interest % 100);

    return 0;
}
```

```
1 interest=Rs 200.00 payment=Rs 2000.00 balance=Rs 8200.00
2 interest=Rs 164.00 payment=Rs 2000.00 balance=Rs 6364.00
3 interest=Rs 127.28 payment=Rs 2000.00 balance=Rs 4491.28
4 interest=Rs 89.83 payment=Rs 2000.00 balance=Rs 2581.11
5 interest=Rs 51.62 payment=Rs 2000.00 balance=Rs 632.73
6 interest=Rs 12.65 payment=Rs 645.38 balance=Rs 0.00
Months: 6
Total interest: Rs 645.38
```

Hand-verified schedule: Rs 10,000, 2% interest, Rs 2,000 repayment

Month	Opening balance	Interest (2%)	Payment	Closing balance
1	10,000.00	200.00	2,000.00	8,200.00
2	8,200.00	164.00	2,000.00	6,364.00
3	6,364.00	127.28	2,000.00	4,491.28
4	4,491.28	89.83	2,000.00	2,581.11
5	2,581.11	51.62	2,000.00	632.73
6	632.73	12.65	645.38	0.00

The final payment is Rs 645.38, not Rs 2,000, because the amount due after interest is only Rs 645.38. Total interest is Rs 645.38 and the loan is repaid in 6 months. The program calculates interest to the nearest paise each month.

Termination and growth investigation

- If repayment is greater than monthly interest, the balance normally decreases and the loan can be repaid.
- If repayment equals monthly interest, the balance does not decrease; the loan never finishes.
- If repayment is smaller than monthly interest, the balance increases; the loan becomes larger.
- The 360-month safety limit prevents an endless run. If reached, the program should clearly report that the loan is not fully repaid rather than falsely claiming success.

As repayment decreases towards the monthly interest, the principal reduction per month becomes smaller and the number of iterations grows. At or below the interest amount, the schedule does not converge to zero. The program should check this condition before starting, or retain the safety limit as a second protection.

Direct formula alternative

For fixed monthly rate r, initial principal P and fixed repayment E, the stated formula estimates the number of months as $n = -\log(1 - r \cdot P/E) / \log(1 + r)$. It takes O(1) time and O(1) space, compared with O(n) monthly iterations for simulation. The formula is valid only when its assumptions match the payment model and the logarithm argument is valid. It may produce a fractional number of months and floating-point rounding affects the final payment. The monthly simulation is easier to audit and naturally handles the smaller final payment; the formula is useful for an estimate, followed by a final-month adjustment.

Q7 – Diagnose, Redesign and Validate

Selected program: the original prime-number trial-division program in Q5. This program has a clear performance limitation when ticket numbers become large.

A. Diagnose the bottleneck

The main work is the repeated remainder operation n % i inside the divisor loop. In the original version, the loop tests every integer from 2 to n − 1. Therefore the number of tests grows almost linearly with n. For the ticket 1,000,003, it can require 1,000,001 divisor tests if the number is prime. The square-root version needs only 999 tests for the same number. The evidence shows that many checks above the square root are unnecessary because any factor above the square root has a matching factor below it.

B. Define the design objective

Objective: reduce the number of divisor checks for a large prime ticket from approximately one million to fewer than one thousand, while preserving the correct prime/non-prime result and using constant extra memory.

C. Design the revised solution

The revised design first handles values below 2, then checks divisibility by 2 once. After that it tests only odd divisors 3, 5, 7, and so on, stopping when i > n / i. This uses two improvements: the square-root property and the fact that every even number greater than 2 is composite. No array or additional data structure is required.

```
int is_prime_optimized(unsigned long n) {
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;

    for (unsigned long i = 3; i <= n / i; i += 2) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
```

Pseudocode:

```
START
  Read n
  If n < 2, report NOT PRIME
  If n = 2, report PRIME
  If n is even, report NOT PRIME
  Set i = 3
  While i <= n / i
      If n mod i = 0, report NOT PRIME and stop
      i = i + 2
  Report PRIME
STOP
#include <stdio.h>
int main(void) {
    int n, i;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 2) {
        printf("NOT PRIME\n");
        return 0;
    }
    if (n == 2) {
        printf("PRIME\n");
        return 0;
    }
    if (n % 2 == 0) {
        printf("NOT PRIME\n");
        return 0;
    }
    i = 3;
```

```
    while (i <= n / i) {
        if (n % i == 0) {
            printf("NOT PRIME\n");
            return 0;
        }
        i = i + 2;
    }
    return 0;
}
```

```c
#include <stdio.h>
int main(void) {
    int n, i;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 2) {
        printf("NOT PRIME\n");
        return 0;
    }
    if (n == 2) {
        printf("PRIME\n");
        return 0;
    }
    if (n % 2 == 0) {
        printf("NOT PRIME\n");
        return 0;
    }
    i = 3;
    while (i <= n / i) {
        if (n % i == 0) {
            printf("NOT PRIME\n");
            return 0;
        }
        i = i + 2;
    }
    return 0;
}
```

```
Enter a number: 2
PRIME
```

D. Validation and trade-offs

Input	Expected result	Optimized divisor checks
1	Not prime	0
2	Prime	0
3	Prime	0
97	Prime	4 (3, 5, 7, 9)
100	Not prime	1 (even check)
7919	Prime	43 odd candidates from 3 through 87
1,000,003	Prime	499 odd candidates from 3 through 999

For 1,000,003, the revised method performs 499 odd-divisor tests after the even check, instead of 999 tests in the square-root version and 1,000,001 in the original $n - 1$ version. It therefore achieves the measurable objective of fewer than 1,000 divisor checks.

The revised method uses $O(\sqrt{n})$ time and $O(1)$ extra space. It is more efficient but slightly more complex because it has special cases for 2 and even numbers. For a very large number of queries up to a known maximum, a sieve may be faster overall but requires $O(N)$ memory. For a small number of isolated ticket checks, the optimized trial-division method is a practical balance.